1 Project Planning & Management

1.1 Project Proposal

- **Overview:** The "AI-Enhanced Data Pipeline for Customer Churn Prediction" project focuses on building an automated, end-to-end data pipeline that integrates machine learning models to identify customers at risk of churning. The system will process raw customer data using Python and SQL, execute a pre-trained AI model to generate predictions, and serve these insights via a deployed service.
- **Objectives:** The primary goals are to extract predictive features from behavioral and transaction datasets, integrate a pre-trained model (such as XGBoost or Random Forest), automate the execution workflow, and provide continuous monitoring of model performance and business trends.
- **Scope:** The project covers the entire data lifecycle. It includes data ingestion, feature extraction via SQL, data cleaning using Python, and the development of model inference scripts. The scope also extends to automating the pipeline using tools like Apache Airflow or Azure Data Factory, deploying the model as a REST API via Flask or FastAPI, and building business intelligence dashboards (e.g., Power BI, Grafana) to visualize churn trends.

1.2 Project Plan

To ensure the project is delivered by the student deadline of 2/20/2026 for the planning phase and meets all subsequent deadlines, the work is divided into four primary milestones:

- **Milestone 1: Data Collection & Feature Engineering**
  - *Deliverables:* Feature dataset (CSV/Database), Python preprocessing scripts, SQL feature extraction scripts.
- **Milestone 2: AI Model Integration**
  - *Deliverables:* Model inference code, model evaluation report detailing accuracy, precision, and recall.
- **Milestone 3: Pipeline Automation & Deployment**
  - *Deliverables:* Workflow automation scripts (DAGs), deployed REST API endpoint, API documentation.
- **Milestone 4: Monitoring & Reporting**
  - *Deliverables:* Monitoring logs, business intelligence dashboard screenshots, final project report and presentation.

1.3 Task Assignment & Roles

To successfully execute this full-stack data engineering and AI project, responsibilities have been distributed among the team members based on the project's milestones:

Team Member	Assigned Role & Responsibilities
Yossef	**Data Engineer:** Responsible for acquiring customer behavioral datasets and writing SQL queries for initial data extraction.
Mohamed	**Data Engineer:** Collaborates on feature extraction, focusing on usage frequency, complaints, and payment history metrics.
Ali	**AI & ML Engineer:** Handles data cleaning and preprocessing in Python, loads the pre-trained model (XGBoost/Random Forest), and develops the core model inference scripts.
Mariam	**Backend Developer:** Responsible for taking the predictive model and deploying the pipeline as a scalable REST API using Flask or FastAPI.
Jana	**Workflow Automation Engineer:** Designs and implements the automated pipeline using Apache Airflow or Azure Data Factory (ADF) to schedule feature extraction and prediction runs.
Mariam Goda	**BI & Monitoring Specialist:** Sets up system logging for prediction results and model drift, and designs the final visualization dashboards in Power BI/Grafana.

Table 1: Task Assignment and Roles

1.4 Risk Assessment & Mitigation Plan

- **Risk:** Poor Data Quality or Missing Values in raw datasets.
  - **Mitigation:** Implement robust Python preprocessing scripts to handle missing data and normalize inputs before they reach the model.
- **Risk:** Pipeline execution failures or bottlenecks.
  - **Mitigation:** Utilize Apache Airflow or Azure Data Factory for automated orchestration with built-in retry mechanisms and dependency management.
- **Risk:** Degradation of AI model performance over time (Model Drift).
  - **Mitigation:** Implement strict logging to track prediction outputs and set up dashboard indicators specifically to monitor model drift metrics.
- **Risk:** REST API downtime preventing business users from getting predictions.
  - **Mitigation:** Thoroughly test end-to-end API responsiveness prior to production and document API usage clearly.

1.5 KPIs (Key Performance Indicators)

To measure the success and reliability of the data pipeline and the AI model, the following metrics will be tracked:

- **Model Performance Metrics:** Maintaining high Accuracy, Precision, and Recall thresholds as defined in the Milestone 2 evaluation report.
- **Pipeline Execution Time:** The total time taken for the automated workflow (Airflow/ADF) to extract data, process it, and run batch predictions.
- **API Response Time:** The latency of the Flask/FastAPI REST API when a request is made for a live churn probability score.
- **System Uptime:** Ensuring the deployed pipeline and dashboard services remain highly available for business reporting.

2 Literature Review

2.1 Feedback & Evaluation

Note: This section is a placeholder reserved for the lecturer's assessment upon project submission. As per the project guidelines, the final evaluation will be based on the final presentation, which must include adherence to the deliverables and the distribution of tasks among team members.

- **Milestone 1 (Data Collection & Feature Engineering):** [Lecturer feedback on data extraction and preprocessing to be inserted here].
- **Milestone 2 (AI Model Integration):** [Lecturer feedback on model validation and accuracy metrics to be inserted here].
- **Milestone 3 (Pipeline Automation & Deployment):** [Lecturer feedback on API responsiveness and workflow automation to be inserted here].
- **Milestone 4 (Monitoring & Reporting):** [Lecturer feedback on dashboard insights and logging to be inserted here].

2.2 Suggested Improvements

Note: This section will be finalized post-evaluation. However, anticipated areas where the pipeline can be enhanced include:

- **Advanced AI Architectures:** Transitioning from standard models (e.g., XGBoost, Random Forest) to Deep Learning approaches for more complex behavioral pattern recognition.
- **Real-Time Stream Processing:** Upgrading the pipeline from batch processing to real-time event streaming (e.g., using Apache Flink or Kafka) to predict churn at the exact moment a customer interacts with the service.
- **Enhanced Dashboard Interactivity:** Expanding the Power BI or Grafana dashboards to include predictive "what-if" scenario testing, allowing business users to simulate how different retention strategies might affect the churn probability.

2.3 Final Grading Criteria

Based on the DEPI documentation guidelines, the breakdown of marks will be evaluated across the following core areas:

- **Documentation:** Clarity, completeness, and adherence to the standard project phases (Planning, Literature Review, Requirements Gathering, and System Analysis).
- **Implementation (Source Code & Execution):** The quality, modularity, and execution of the Python preprocessing scripts, SQL feature extraction, AI model inference code, and automated workflow scripts (Airflow or Azure Data Factory).
- **Testing & Quality Assurance:** Successful validation of prediction results against a test dataset, completion of test cases, bug reports, and end-to-end API responsiveness.
- **Final Presentation:** The effectiveness of the final project presentation, clarity of the final project report, and the team's ability to demonstrate the work completed while identifying each member's responsibility for their assigned tasks.

3 Requirements Gathering

3.1 Introduction

This section provides a comprehensive specification of the system requirements for the "AI-Enhanced Data Pipeline for Customer Churn Prediction." The purpose of this section is to formally define the expected behavior, performance, and constraints of the system to ensure alignment between stakeholders and the development team. The requirements outlined here serve as a foundation for system design, implementation, testing, and evaluation.

3.2 Stakeholder Analysis

The successful delivery of the system depends on addressing the needs of multiple stakeholders:

- **Business Analysts:** Require accurate and interpretable churn predictions to support customer retention strategies and decision-making processes.
- **Data Engineers:** Responsible for designing, implementing, and maintaining scalable data pipelines and ensuring data integrity.
- **AI/ML Engineers:** Focus on model integration, validation, optimization, and monitoring of predictive performance.
- **System Administrators:** Ensure system reliability, availability, and performance through monitoring and maintenance.
- **End Users (Decision Makers):** Interact with dashboards and APIs to retrieve insights and predictions.

- **Project Supervisor:** Evaluates the project deliverables, technical quality, and adherence to academic and engineering standards.

3.3 Functional Requirements

The system shall implement the following core functionalities:

- **Data Ingestion:**
  - The system shall support ingestion of structured and semi-structured data from multiple sources (CSV files, relational databases, APIs).
  - The ingestion process shall be automated and scheduled via workflow orchestration tools.
- **Data Preprocessing:**
  - The system shall perform data cleaning, including handling missing values, removing duplicates, and correcting inconsistencies.
  - The system shall normalize numerical features and encode categorical variables.
- **Feature Engineering:**
  - The system shall extract relevant features using SQL queries and Python transformations.
  - Derived features such as customer lifetime value, engagement frequency, and complaint ratios shall be generated.
- **Model Integration and Inference:**
  - The system shall load a pre-trained machine learning model (e.g., XGBoost or Random Forest).
  - The system shall generate churn probability scores for each customer.
  - The system shall classify customers into churn risk categories (e.g., low, medium, high).
- **Data Storage:**
  - The system shall store processed data and prediction results in a persistent storage system (database or data warehouse).
- **API Deployment:**
  - The system shall expose prediction services via a RESTful API using Flask or FastAPI.
  - The API shall support real-time prediction requests and return results in JSON format.
- **Visualization and Reporting:**

  - The system shall provide dashboards to visualize churn trends, key metrics, and predictions.
  - The dashboards shall support filtering, aggregation, and time-based analysis.

- **Logging and Monitoring:**
  - The system shall log pipeline execution details, prediction outputs, and system errors.
  - The system shall track model performance metrics over time to detect drift.

3.4 Non-Functional Requirements

The system shall satisfy the following quality attributes:

- **Performance:**
  - Batch processing pipelines shall complete within 5 minutes for datasets up to 100,000 records.
  - API response time shall not exceed 2 seconds per request.

- **Scalability:**
  - The system shall handle increasing data volumes through modular and distributed architecture.

- **Reliability:**
  - The system shall implement retry mechanisms and failure recovery strategies in workflow orchestration tools.

- **Accuracy:**
  - The predictive model shall maintain a minimum accuracy of 90%, with balanced precision and recall.

- **Availability:**
  - The system shall ensure at least 95% uptime for API and dashboard services.

- **Security:**
  - The system shall ensure secure handling of data, including controlled API access and data privacy considerations.

- **Usability:**
  - The dashboards shall provide clear and intuitive visualizations for non-technical users.

3.5 Data Requirements

- Data shall include customer demographics, transaction history, usage behavior, and complaint records.
- Data shall be stored in structured formats such as relational databases or CSV files.
- Data preprocessing shall include:
  - Handling missing values using imputation techniques.
  - Encoding categorical variables (e.g., one-hot encoding).
  - Feature scaling (normalization or standardization).
- The dataset shall be split into training, validation, and testing subsets.
- Class imbalance shall be addressed using techniques such as resampling or class weighting.

3.6 System Requirements

- **Programming Languages:** Python, SQL
- **Machine Learning Libraries:** Scikit-learn, XGBoost
- **Data Processing Libraries:** Pandas, NumPy
- **API Frameworks:** Flask or FastAPI
- **Workflow Orchestration:** Apache Airflow or Azure Data Factory
- **Visualization Tools:** Power BI, Grafana
- **Execution Environment:** Local machine, cloud platform, or Kaggle environment

3.7 Constraints

- Limited availability of high-quality labeled churn datasets.
- Time constraints due to academic deadlines.

3.8 Assumptions

- The available dataset is representative of real-world customer behavior.
- The selected machine learning model is suitable for churn prediction.
- All required tools and frameworks are accessible and properly configured.

4 System Analysis & Design

4.1 Problem Statement & Objectives

- **Problem Statement:** Businesses face significant revenue loss when customers unexpectedly discontinue services. Identifying high-risk customers manually is inefficient and prone to error. There is a need for a unified, automated data pipeline capable of ingesting raw transaction data, extracting behavioral features, and predicting churn probability using machine learning models to enable proactive retention strategies.

- **Project Goals:**

  1. Design and implement a scalable ETL (Extract, Transform, Load) pipeline.
  2. Train and integrate an AI model capable of scoring churn risk.
  3. Deploy the system via a REST API to serve predictions to business dashboards.

- **Use Case Diagram & Descriptions:**

  – *Actors:* Data Engineer, ML Engineer, Business Analyst, Automated Scheduler (e.g., Airflow).

  – *Interactions:* The Scheduler triggers the ETL pipeline; the Data Engineer monitors data health; the ML Engineer updates the prediction model; the Business Analyst requests churn insights via the API/Dashboard.

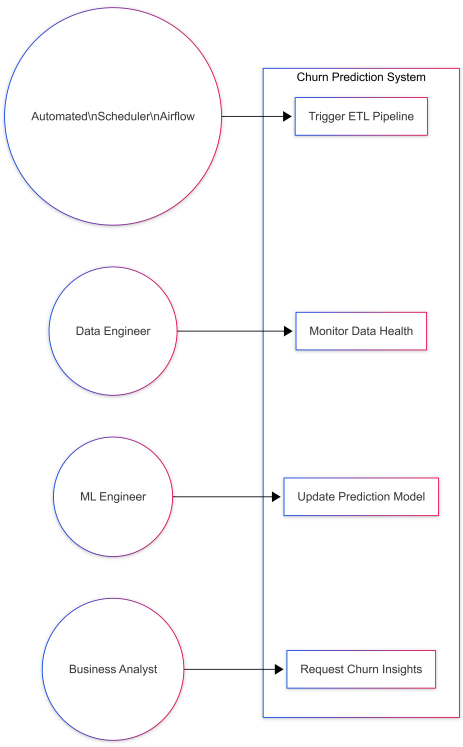

Figure 1: Use Case Diagram: System Actors and Interactions

- **Software Architecture:** The system utilizes a **Modular Pipeline Architecture**. It decouples data processing (Python/SQL scripts), task orchestration (Apache Airflow/ADF), model serving (Flask/FastAPI), and the presentation layer (Power BI/Grafana).

4.2 Database Design & Data Modeling

- **ER Diagram (Entity-Relationship Diagram):** The database structure models the core business entities required for churn analysis. Key entities include *CustomerProfile*, *SubscriptionDetails*, *UsageLogs*, and *ChurnPredictions*.

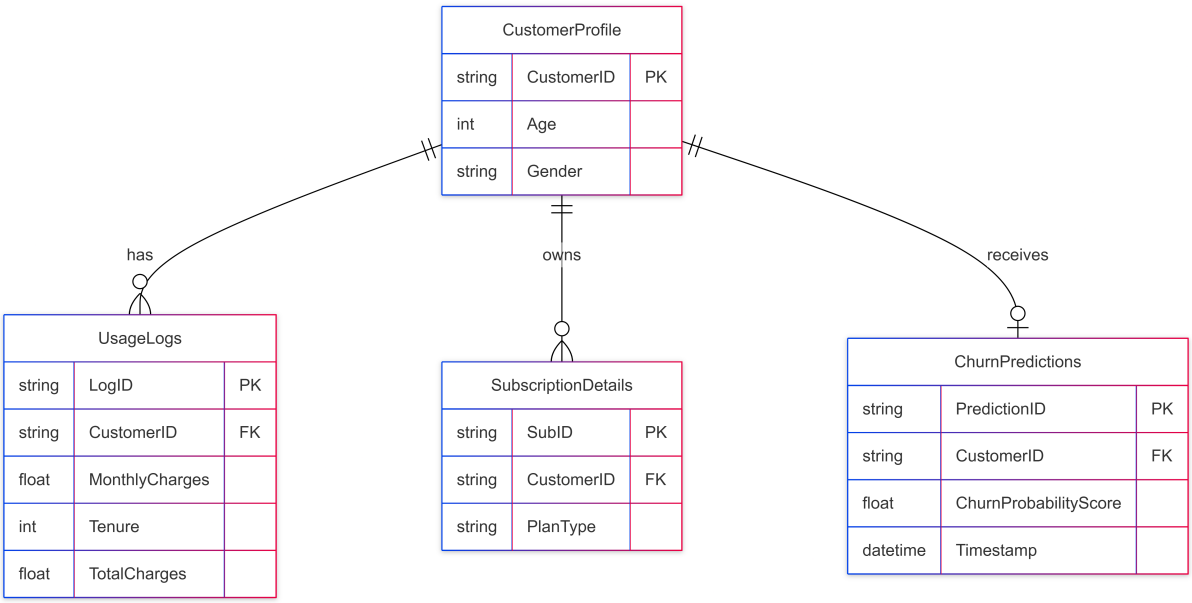

Figure 2: Entity-Relationship Diagram (ERD)

- **Logical & Physical Schema:**
  - **Customer Table:** Primary Key (CustomerID), Demographics (Age, Gender).
  - **Usage Table:** Foreign Key (CustomerID), MonthlyCharges, Tenure, TotalCharges.
  - **Prediction Table:** Foreign Key (CustomerID), ChurnProbabilityScore, Timestamp.
  - Data is normalized to 3NF to prevent redundancy before being denormalized into a flat feature matrix for model ingestion.

4.3 Data Flow & System Behavior

- **DFD (Data Flow Diagram):**
  - *Level 0 (Context):* Raw customer data enters the system → Churn Pipeline processes data → Churn insights are delivered to the Business Analyst.

– *Level 1:* Highlights internal sub-processes: Data Ingestion → SQL Feature Extraction → Python Cleaning → ML Inference → Database Storage → API Output.

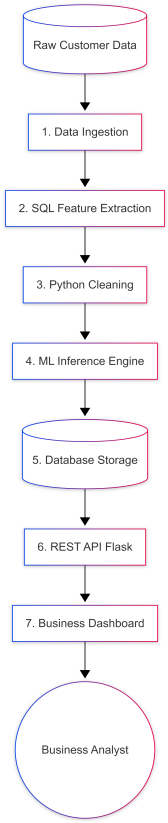

Figure 3: Data Flow Diagram (Level 1)

- **Sequence Diagram:** Maps the chronological flow of a prediction request. A user interacts with the Dashboard → Dashboard sends a REST GET/POST request to the Flask API → API queries the Database/Model → API returns JSON response → Dashboard updates visually.

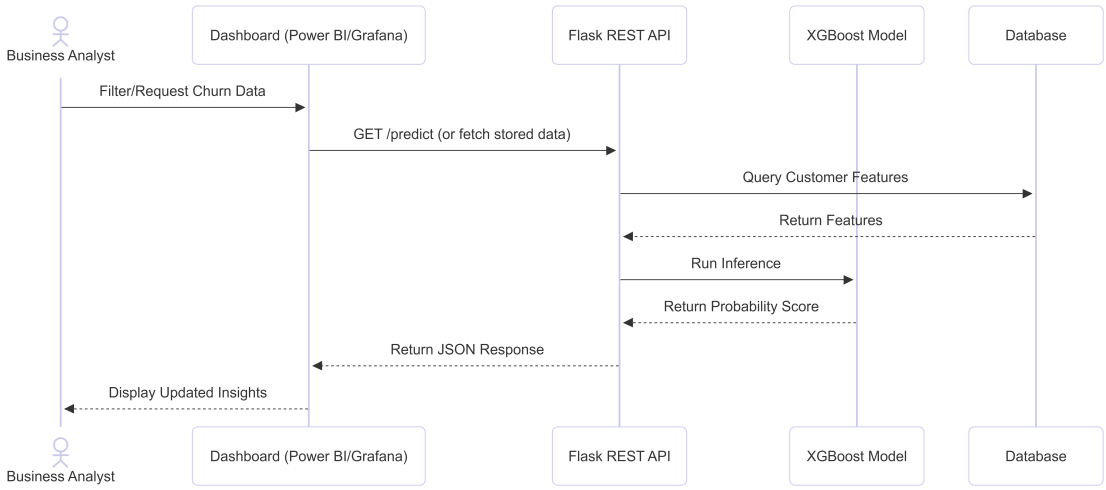

Figure 4: Sequence Diagram: Prediction Request Workflow

- **Activity Diagram:** Visualizes the workflow of the automated ETL job (Triggered by Airflow → Check source data → Clean data → Run Batch Prediction → Log success/failure).

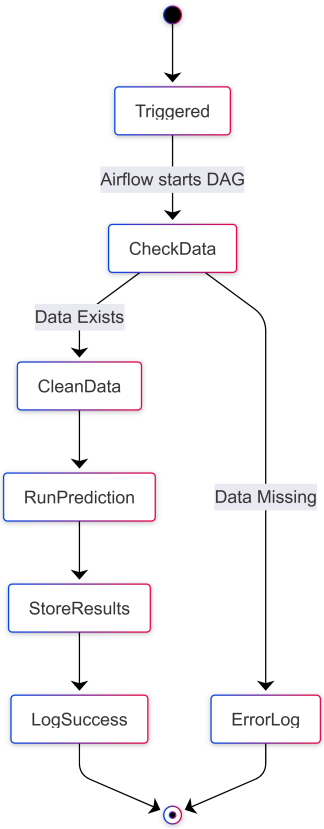

Figure 5: Activity Diagram: ETL Workflow Execution

- **State Diagram:** Represents the state transitions of a customer profile (e.g., Active, At-Risk, Churned, Retained).

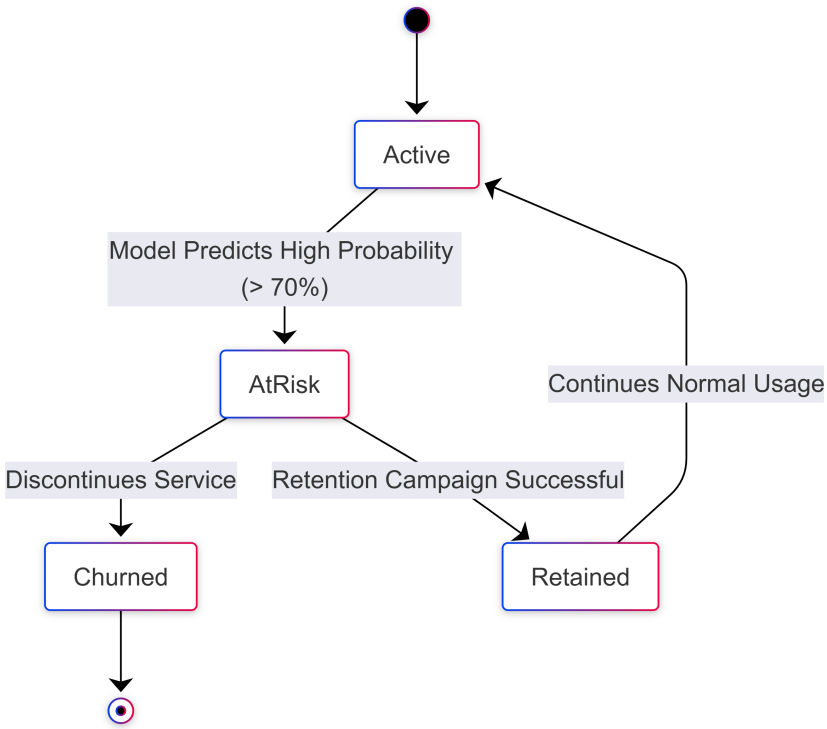

Figure 6: Customer State Transition Diagram

- **Class Diagram:** Maps the backend object-oriented structure (e.g., `DataProcessor` class, `ModelPredictor` class, `APIController` class).

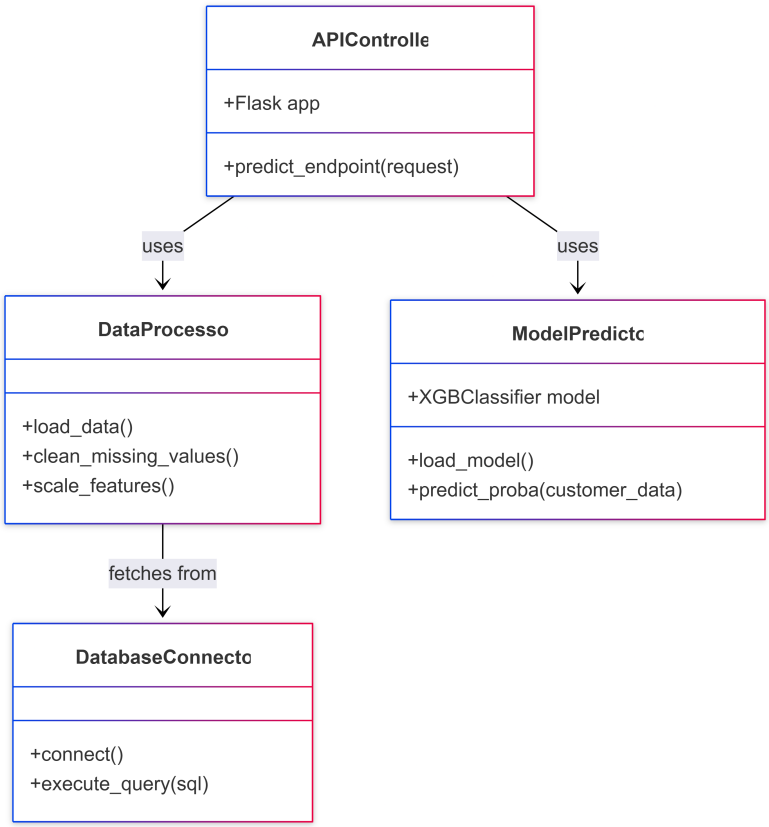

Figure 7: System Class Diagram

4.4 UI/UX Design & Prototyping

- **Wireframes & Mockups:** The visual interface consists of an analytical dashboard. The main screen features a high-level KPI banner (Total Customers, Overall Churn Rate), followed by a sortable table flagging "High Risk" customers, and a feature-importance chart explaining the AI's reasoning. *(Note: Final UI Mockup images will be inserted post-development).*
- **UI/UX Guidelines:**
  - *Color Scheme:* Use standard alert colors (Red for high-risk churn, Green for safe/retained).
  - *Accessibility:* Ensure high contrast between text and background.
  - *Typography:* Use clean, sans-serif fonts for data readability (e.g., Roboto or Open Sans).

4.5 System Deployment & Integration

- **Technology Stack:**
  - *Data Engineering:* SQL, Pandas, NumPy.
  - *Orchestration:* Apache Airflow or Azure Data Factory.
  - *Machine Learning:* Scikit-learn, XGBoost.

- *Backend/API:* Python, Flask / FastAPI.
- *Frontend/BI:* Power BI or Grafana.

- **Deployment Diagram:** Details how software components map to hardware or cloud services. For example: An execution server running Airflow, a database server (or cloud SQL instance) holding processed data, and a web server hosting the Flask API.

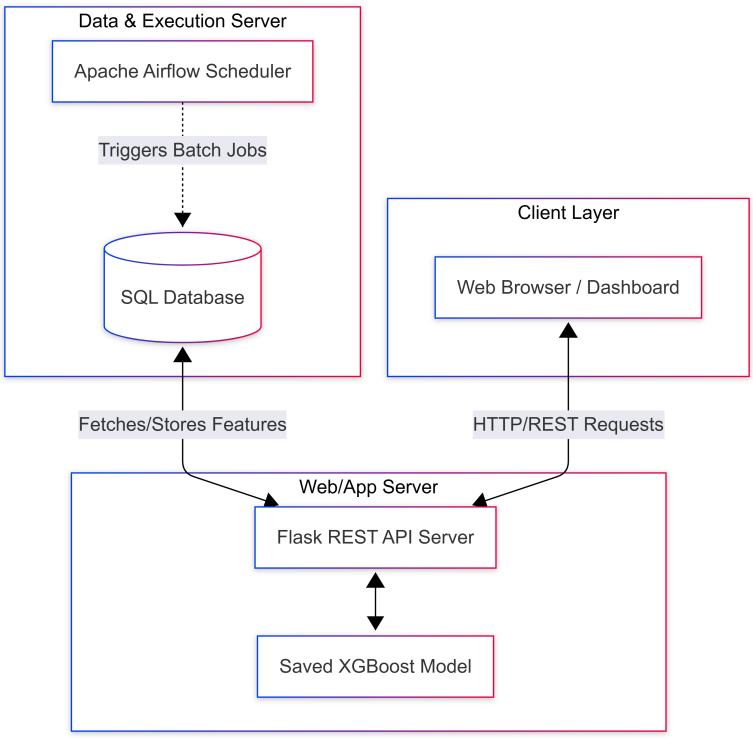

Figure 8: System Deployment and Architecture Diagram

- **Component Diagram:** Illustrates the modular relationships between the Data Ingestion Module, the ML Inference Engine, the Logging/Monitoring Component, and the REST API interface. *(Note: Final Component Diagram will be inserted based on the finalized architecture).*